



Data Observability & Lineage

AICB-P2T2 · Ngày 27 · Chương 6: Tổng Hợp

Giảng viên

VinUniversity · Phase 2 · Track 2 · Tuần 6

HÃY SUY NGHĨ...

*“Pipeline chạy thành công nhưng data sai – làm sao bạn biết? **Case study:** Một team phát hiện model accuracy giảm 15% – sau 3 ngày mới biết up-stream data bị schema change. Data observability phát hiện trong 60 giây.”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Data Observability vs Pipeline Monitoring
2. Great Expectations: Suites & Checkpoints
3. Monte Carlo & Anomaly Detection
4. dbt Tests: Unit & Integration
5. SLO Engineering cho Data & AI
6. Incident Response cho Data Systems
7. Live Demo: Incident Detection
8. Labs: Data Observability Implementation

Sau buổi học này, bạn sẽ:

1. Master data observability với Great Expectations + Monte Carlo
2. Thiết kế advanced Grafana dashboards cho data quality
3. Implement SLO engineering cho data & AI services
4. Xây dựng incident response workflow cho data incidents

Data observability concepts (20 min) → Great Expectations (45 min) → Monte Carlo / dbt tests (30 min) → SLOs (30 min) → Demo & Labs

GE checkpoint suite + Monte Carlo-style anomaly detection + SLO dashboard

- Great Expectations Checkpoint chạy trong Airflow DAG
- Z-score anomaly detection script với Slack alert
- 3 SLOs cho data platform + Grafana dashboard với error budget panel
- Incident response runbook document

Job Airflow báo **Success**. Logs sạch. CPU bình thường. Nhưng dashboard doanh thu sai 40%.

Ba lỗi âm thầm phổ biến:

- Source API bị rate-limit → chỉ lấy được 10% records
- Cột price đổi từ USD sang VND không thông báo
- Join bảng dimension fail ngầm vì customer_id đổi kiểu dữ liệu

Tại sao pipeline không phát hiện?

- Pipeline chỉ quan tâm *tiến trình có chạy xong không*
- Không quan tâm *kết quả có đúng không*
- Airflow success = job finished, **không phải** data correct

Kết luận: Lỗi nguy hiểm nhất không phải crash – mà là **silent failure**: hệ thống chạy bình thường nhưng dữ liệu sai. AI/ML và analytics chết vì silent bad data nhiều hơn vì service down.

Nhìn từ góc hạ tầng:

- Job có chạy xong không?
- Chạy mất bao lâu?
- Có lỗi kỹ thuật không?
- CPU, memory có ổn không?

⇒ Trả lời câu hỏi: **“Máy có chạy không?”**

Nhìn từ góc chất lượng dữ liệu:

- Data có **đúng** không?
- Data có **mới** (fresh) không?
- Data có **đầy đủ** không?
- Data có **nhất quán** không?

⇒ Trả lời câu hỏi: **“Data có đáng tin không?”**

Cả hai đều cần thiết và bổ sung cho nhau. Monitoring bảo vệ hạ tầng. Observability bảo vệ *niềm tin* vào dữ liệu – thứ mà dashboard, ML model, và AI service phụ thuộc vào mỗi ngày.

Freshness

Dữ liệu có cập nhật đúng cadence?

Volume

Số records có bất thường, đột biến?

Distribution

Phân phối giá trị có thay đổi?

Schema

Cột bảng có bị đổi tên/kiểu?

Lineage

Khi incident, bảng nào bị kéo theo?

Mỗi trụ cột = một chiều rủi ro: Freshness phát hiện delay, Volume phát hiện mất data, Distribution phát hiện giá trị bất thường, Schema phát hiện breaking changes, Lineage giúp trace root cause.

Chi phí thực tế: 10+ giờ/tuần data downtime, \$15M/năm thiệt hại từ bad data (Gartner) – observability đánh đổi đầu tư nhỏ để tránh thiệt hại lớn.



Hiểu từng Level để biết mình đang ở đâu: **L0** – check bằng mắt, user báo mới biết (nguy hiểm nhất). **L1** – rules rõ ràng, tự động, bắt *known problems*. **L2** – học từ lịch sử, phát hiện *unknown unknowns*. **L3-4** – predictive & self-healing, cần đầu tư lớn hơn.

Hầu hết teams ở **Level 0-1**. Mục tiêu khóa này: đạt **Level 2** và hiểu lộ trình lên Level 3-4.

Dữ liệu text, ảnh, embedding không có schema rõ ràng – không thể check null rate hay value range trực tiếp. Cần cách tiếp cận khác.

Giải pháp: trích xuất *derived features* rồi monitor chúng:

- **Embedding drift:** cosine similarity giữa batch mới vs baseline giảm → retrieval quality suy giảm
- **Phân phối token:** độ dài text đối đột ngột → upstream source có vấn đề
- **Chất lượng ảnh:** blur score, resolution distribution đổi bất thường
- **RAG metrics:** chunk count, retrieval hit rate, answer length distribution

Không monitor raw content.

Monitor **measurable features** trích xuất từ content.

Công cụ: KL divergence, KS test, cosine similarity.

Tư duy: “biến thứ không đo được thành thứ đo được”.

Mọi data engineer đều có *giả định ngầm* về dữ liệu – nhưng chúng chỉ nằm trong đầu, không được kiểm tra tự động. Khi dữ liệu sai, không ai biết cho đến khi user phàn nàn.

Ví dụ giả định ngầm trong hệ thống:

- Email của user không bao giờ null
- Tuổi người dùng nằm trong khoảng 0-150
- Cột status chỉ có 3 giá trị hợp lệ
- Mỗi ngày có ít nhất 50K đơn hàng

Great Expectations (GX) biến chúng thành:

- Kiểm tra tự động chạy được bằng máy
- Lưu trong Git cùng pipeline code
- Tái sử dụng trên dev/staging/prod
- Báo cáo HTML tự động cho stakeholders

Phép so sánh: Expectation = assert; suite = file test cho một bảng; checkpoint = bước CI chạy cả bộ test.

- **Completeness:** cột critical không null, row count > 0
- **Uniqueness:** primary key không trùng
- **Validity:** giá trị trong range hoặc đúng format
- **Consistency:** tổng chi tiết khớp tổng aggregate
- **Freshness-like:** ngày mới nhất không quá cũ

Hard fail – block pipeline ngay:

- Primary key bị duplicate
- Source table hoàn toàn rỗng
- Thiếu cột bắt buộc

Soft fail – chỉ cảnh báo, tiếp tục:

- Null rate tăng nhẹ
- Distribution hơi lệch
- Text description bất thường

Nguyên tắc thiết kế: Expectation quá lỏng (age 0--999) thì vô dụng. Quá chặt thì tạo false alarm liên tục. Hãy gắn từng rule với *business semantics* thật, không phải kỹ thuật thuần túy.

```
import great_expectations as gx
context = gx.get_context()
suite = context.add_expectation_suite("users_suite")

suite.add_expectation(
    gx.expectations.ExpectColumnValuesToNotBeNull(
        column="email"
    )
)
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToBeBetween(
        column="age", min_value=0, max_value=150
    )
)
```

- `get_context()`: khởi tạo GX workspace
- `add_suite`: tạo bộ rules cho asset
- Expectation 1: email phải luôn có giá trị (completeness)
- Expectation 2: tuổi nằm trong khoảng hợp lý (validity)

Checkpoint = Suite + Datasource + Actions

- **Suite:** bộ rules đã định nghĩa cho asset
- **Datasource:** batch dữ liệu thực tế cần validate
- **Actions:** việc cần làm *sau khi* chạy validation

Actions phổ biến khi fail:

- SlackNotificationAction: gửi alert tức thì
- StoreEvaluationParameters: lưu metrics để track
- Block pipeline: không để data xấu đi tiếp downstream

Source / Ingest



GX Checkpoint



fail → Slack pass ↓

Transform (dbt)



Serving Table

Tự động sinh HTML report để stakeholder xem pass/-

fail mà không cần đọc code.

Câu hỏi: “Data có vi phạm rule đã biết không?”

Ví dụ: email null, price âm, status ngoài tập cho phép

Ưu điểm: rõ ràng, deterministic, không tranh cãi

Nhược điểm: chỉ bắt được thứ đã nghĩ ra trước

Câu hỏi: “Dữ liệu hôm nay có cư xử lạ không?”

Ví dụ: row count giảm 50%, phân phối đột ngột đổi

Ưu điểm: bắt được *unknown unknowns*

Nhược điểm: có false positives, cần người re-view

Phép so sánh trực quan: Rules-based giống kiểm tra cửa ra vào với danh sách điều kiện. Anomaly detection giống camera an ninh nhìn hành vi – không cần biết rule cụ thể, chỉ cần thấy “hôm nay trông khác hôm thường”.

Production tốt nhất: kết hợp cả hai lớp. Không thay thế lẫn nhau.

- Kết nối warehouse → tự động monitor 200+ metrics
- ML-based anomaly detection không cần cấu hình thủ công
- Incident timeline: dùng lineage tìm root cause trong vài phút
- Alert qua Slack, PagerDuty, email

⇒ Đại diện cho loại **platform observability**: mua SaaS thay vì tự xây từ đầu

- ydata-profiling: profile metrics và statistics
- Z-score: $\text{abs}(\text{current} - \text{mean}) / \text{std} > 3$
- Time-series: Prophet dự đoán expected value
- Alert khi actual $> 3\sigma$ deviation
- Export metrics sang Prometheus/Grafana

⇒ Linh hoạt hơn, nhưng cần engineering & maintenance liên tục

```
import numpy as np

def detect_anomaly(current_value, history, threshold=3):
    mean = np.mean(history)
    std = np.std(history)
    if std == 0:
        return False, 0.0
    z_score = abs(current_value - mean) / std
    return z_score > threshold, z_score

daily_counts = [10200, 10150, 10300, 10180, 10250]
today_count = 5100 # à gim 50%!
anomaly, score = detect_anomaly(today_count, daily_counts)
# anomaly=True, score=7.2 -> ALERT!
```

- mean: hành vi trung bình lịch sử
- std: độ biến động bình thường
- z_score: lệch bao nhiêu *standard deviation*
- $> 3\sigma$: rất bất thường → alert

- Metric có seasonality (cuối tuần vs ngày thường)
- Sự kiện đặc biệt (flash sale, chiến dịch)
- Lịch sử quá ngắn < 14 ngày
- ⇒ Cần Prophet hoặc segment baseline

- Biết rõ giá trị hợp lệ là gì
- Cần hard fail – block pipeline ngay
- Muốn kết quả deterministic, rõ ràng
- Ví dụ: primary key, null rate critical, accepted values

- Muốn bắt pattern bất thường chưa được viết rule
- Metric có lịch sử đủ dài để học baseline
- Chấp nhận false positives và có người review
- Ví dụ: row count, null rate trend, embedding drift

Alert quá nhạy → nhiều false positives → team mệt mỏi, mất tin tưởng vào alert system.

Alert quá lỏng → bỏ sót anomaly thật → data xấu đi sâu vào downstream mà không ai hay.

Cần **tuning liên tục** và **human review** – anomaly detection không thể hoàn toàn thay con người.

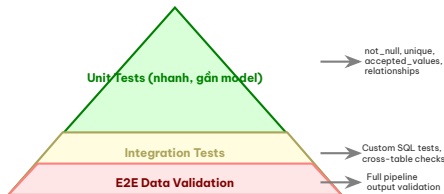
Logic business sống trong SQL – một join sai hoặc filter sai thường không crash gì cả, data vẫn ra nhưng *sai hoàn toàn*

Ví dụ lỗi âm thầm trong dbt:

- Join bảng orders với customers bằng key sai → doanh thu bị inflate lên
- Filter nhầm status = 'completed' bỏ sót đơn hàng → báo cáo thấp hơn thực tế
- SCD logic sai → nhiều version active cùng lúc cho cùng 1 customer

dbt tests là lớp bảo vệ sát nhất:

- Sống cạnh SQL model, chạy cùng dbt build
- Phát hiện ngay sau khi transform xong
- Trước khi serving table xuống downstream
- Không cần tool bên ngoài thêm



Nguyên tắc kim tự tháp: Càng lên cao (Unit), test càng nhanh và gần model – chạy liên tục mỗi lần build. Càng xuống dưới (E2E), phạm vi rộng nhưng tốn kém hơn. **Đừng bỏ Unit tests rồi kỳ vọng E2E cứu hết.**

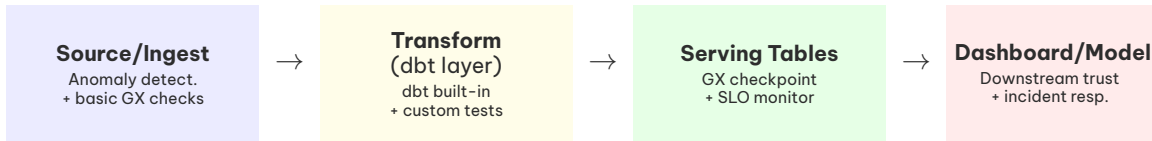
- `not_null` – cột critical không có NULL
- `unique` – primary key không bị trùng
- `accepted_values` – chỉ chấp nhận tập giá trị định sẵn
- `relationships` – foreign key phải tồn tại ở bảng cha

⇒ Miễn phí, nhanh, thiết yếu – chạy mỗi dbt test

- Custom SQL: query trả 0 rows = pass, có rows = fail
- `dbt-expectations`: port mindset GX vào dbt
- **Elementary**: observability quanh dbt – trend, anomaly, dashboard

Ví dụ custom test:

“Không có user nào vừa inactive vừa có subscription active”



Không có tool nào làm hết mọi thứ – mỗi tool có chỗ đứng riêng:

- **GX:** validation tổng quát, report HTML, dùng tốt ở boundary giữa layers
- **dbt tests:** sống cạnh SQL model, rất tự nhiên trong analytics engineering workflow
- **Anomaly detection:** bắt pattern chưa biết, cần baseline lịch sử, cần human review
- **SLO + Incident:** quản lý reliability theo thời gian, ứng phó khi có sự cố

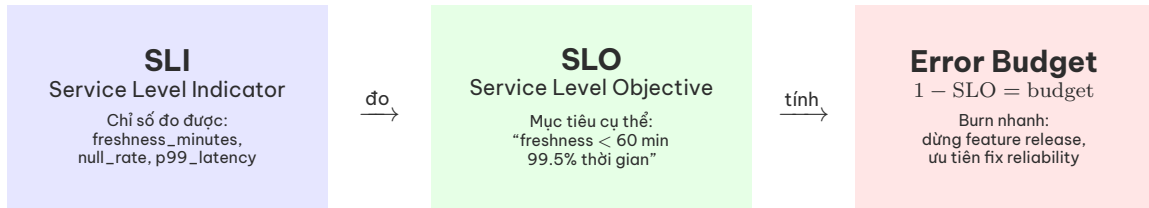
Dữ liệu cũng có *user* – dashboard, ML model, AI service đều phụ thuộc vào data. Khi data không đáp ứng kỳ vọng, downstream user bị ảnh hưởng.

Kỳ vọng thực tế của data users:

- Dashboard CEO phải có data trước 8:00 sáng
- Feature table cho fraud model không stale quá 30 phút
- Null rate ở cột billing phải gần bằng 0
- RAG indexing cập nhật tài liệu mới trong 30 phút

SLO biến kỳ vọng thành cam kết đo được:

- Không còn tranh luận “chắc vẫn ổn”
- Có số liệu rõ ràng để ưu tiên
- Khi breach thì biết cần làm gì tiếp theo
- Tạo văn hóa: reliability là trách nhiệm team, không phải may rủi



Ví dụ cụ thể: SLO = 99.5% freshness < 60 phút $\Rightarrow$ Error budget = 0.5% = **3.6 giờ/tháng** được phép stale.

Ba câu hỏi nền tảng: (1) Ta đo cái gì? $\rightarrow$ SLI (2) Ta muốn tốt tới mức nào? $\rightarrow$ SLO (3) Khi xấu thì làm gì khác? $\rightarrow$ Error budget policy

Burn rate – tốc độ tiêu hao error budget:

- **Fast burn:** đốt 2% budget/giờ → P0 alert, phản ứng ngay
- **Slow burn:** đốt 5% budget/6h → P1 alert, điều tra trong ca
- Burn rate phát hiện sớm hơn chờ breach cuối tháng

SLO Dashboard nên hiển thị:

- Giá trị SLI hiện tại vs target SLO
- Remaining error budget (%)
- Burn rate ngắn hạn (1h) và dài hạn (7 ngày)

SLO buộc team **ưu tiên reliability trước feature mới**.

Budget còn nhiều → release nhanh được.
Budget đang cháy → dừng release, tập trung fix.

SLO không chỉ là số học – nó là **cơ chế quản trị** quyết định khi nào ưu tiên stability.

SLI tốt: đo được tự động, gắn với trải nghiệm downstream user thật sự, ổn định về định nghĩa theo thời gian.

SLI kém: “dashboard trông có vẻ ổn”, “model có vẻ đang chạy tốt”

Ví dụ SLI phù hợp:

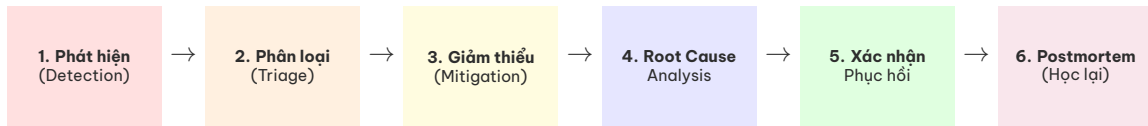
- freshness_minutes: phút kể từ lần update cuối
- null_rate: tỷ lệ null của cột critical
- p99_latency: latency inference API
- schema_violations: số lần schema drift

SLO càng cao → chi phí engineering, alerting, on-call, redundancy càng lớn.

Chỉ đặt SLO gần 100% cho **critical contract** như billing, AI serving trong production.

SLO nên phản ánh **business criticality thật**, không phải kỳ vọng lý tưởng.

SLA vs SLO: SLO là mục tiêu nội bộ để vận hành. SLA là cam kết chính thức với khách hàng, thường nghiêm ngặt hơn và có hậu quả pháp lý.



Tại sao cần quy trình rõ ràng?

Alert mà không ai phản hồi thì vô dụng. Có người phản hồi nhưng không có runbook thì chậm. Có runbook nhưng không có severity thì hỗn loạn.

Observability chỉ có giá trị khi team biết phản ứng – detection là hiệp 1, operations mới là những hiệp còn lại tạo ra giá trị thực sự.

Mức	Mô tả	Thời gian phản ứng	Ví dụ
P0	Hệ thống ngừng hoạt động	5 phút	Pipeline dừng, không có data nào chạy
P1	Dữ liệu sai	30 phút	Giá trị sai ở serving table, model lỗi
P2	Chất lượng giảm sút	2 giờ	SLO bị vi phạm, freshness chậm
P3	Vấn đề nhỏ	Ngày làm việc tiếp theo	Thiếu tài liệu, cảnh báo thấp

- **PagerDuty:** định tuyến on-call, escalation
- **Rundeck:** chạy diagnostic scripts tự động
- **Slack war room:** phối hợp realtime

Service down thì dễ thấy ngay. Data sai có thể âm thầm ảnh hưởng dashboard, billing, model trong nhiều giờ hoặc nhiều ngày mà không ai biết – thiệt hại âm thầm và sâu hơn.

Runbook = hướng dẫn thao tác chuẩn khi có incident:

1. Xác nhận incident và xác định severity
2. Kiểm tra upstream ingestion có đang chạy không
3. So sánh row count source vs destination
4. Kiểm tra schema changes trong 24 giờ gần nhất
5. Dùng lineage: asset downstream nào bị ảnh hưởng?
6. Quyết định: rerun, rollback, hay suppress publication
7. Verify recovery và thông báo stakeholders

- Giảm thời gian chẩn đoán khi đang áp lực
- Giảm sai sót do stress và thiếu ngủ
- Junior engineer xử lý được P2/P3
- Tạo baseline để cải thiện sau postmortem

Dashboard sai → schema đổi → thiếu schema check → source contract mơ hồ → ownership không rõ ràng. Hỏi *tại sao* liên tiếp đến khi thấy systemic cause.

Chủ động inject failure có kiểm soát để test hệ thống và team:

- Kill Airflow worker giữa task đang chạy
- Inject schema change đột ngột vào upstream
- Corrupt một phần source data
- Simulate network partition giữa components

Game day (diễn tập định kỳ): luyện phản xạ trong môi trường an toàn thay vì học lần đầu trên incident thật.

- Detection → Triage → Mitigation
- Root Cause → Verify → Postmortem

Không phải: “ai đã làm sai” → đổ lỗi cá nhân

Đúng hơn: “tại sao hệ thống để điều này xảy ra?” → fix systemic issue

Postmortem tốt luôn có:

- Timeline đầy đủ sự kiện
- 5 Whys root cause analysis
- Action items có owner và deadline cụ thể

Blameless $\neq$ không có accountability. Nó là cách học thật sự từ incident để ngăn tái diễn.

1. **Demo 1:** Inject schema change vào upstream data → GE checkpoint fails → Slack alert trong 60 giây
2. **Demo 2:** Inject volume anomaly (10% of normal) → Z-score detection → PagerDuty alert
3. **Demo 3:** dbt test failure → lineage graph identify upstream source of corruption
4. **Demo 4:** SLO dashboard – show error budget consumption, burn rate alert kích hoạt
5. **Resolution flow:** alert → runbook → auto-diagnostic → root cause → fix → verify

Mục tiêu: Data Observability Implementation

Deliverable: Setup GX Suite với Profiler; build Checkpoint tích hợp Airflow DAG; implement Z-score anomaly detection cho 5 key metrics với Slack alerts; define 3 SLOs và build Grafana SLO dashboard.

Thời gian: 2.5h

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 Data observability $\neq$ pipeline monitoring – cần cả hai, focus khác nhau. Pipeline succeeded không có nghĩa data đúng.
- 2 SLOs buộc team prioritize reliability over features – cultural shift quan trọng hơn tooling.
- 3 Automated anomaly detection phải có human review – false positives cần training models over time.

Ngày 28: Integration Workshop – Full Platform Demo

“Tích hợp toàn bộ infrastructure stack, demo end-to-end platform, hoàn thành Milestone 3”

- Hoàn thành Lab 27: Data Observability Implementation
- Review toàn bộ components từ N16–N27
- Chuẩn bị Milestone 3 demo script

Hỏi & Đáp

Câu hỏi nào về data observability, Great Expectations, SLOs, hay incident response?



Cảm ơn!

AICB-P2T2 · Ngày 27

Data Observability & Lineage

lms.vinuni.edu.vn · Slide & template trên LMS